CLINIC MANAGEMENT SYSTEM

Database Design & Implementation

Mohamed Ahmed Aref	241001386
Sara Khalil Mohamed	241000394
Jana Hosam AbdelWahab	241000888
Hajer Hossam Abdulbaki	241000988
Adham Mohammed AlDakrany	241002505

Academic Project Defense • University Database Course • May 2026

Presentation Agenda

01 Introduction & Problem Definition

02 System Analysis & Design

03 Architecture & Implementation

04 Database Design & Functionality

05 Workflow, Testing & Conclusion

Problem Statement

- Healthcare facilities rely on manual, paper-based processes
- Appointment scheduling is error-prone and slow
- No centralised view of patients, doctors, or departments
- Double-booking conflicts cannot be automatically detected
- No analytical insights into diagnosis trends or revenue
- Data retrieval is inconsistent and time-consuming
- Lack of organisational hierarchy visibility
- Difficult to maintain record integrity across entities

Solution: A full-stack Clinic Management System (CMS) with a relational database at its core.

System Objectives

- Centralise all patient demographic data in a single relational database
- Automate appointment scheduling with overlap prevention logic
- Provide analytical insights: diagnosis trends & total revenue aggregation
- Establish clear organisational hierarchy linking doctors, clinics, and departments
- Offer a modern, responsive web dashboard with real-time CRUD operations

System Scope & Users

Scope

- Single healthcare facility
- 5 core entities managed
- Web-based (browser access)
- MySQL relational backend
- 20 medical departments

System Users

Admin / Staff

Manage all entities, view analytics

Doctor

View scheduled appointments

Receptionist

Register patients, book appointments

Department Mgr

Manage clinics and departments

Research Questions

RQ1	How can a relational database model support the complex scheduling needs of a multi-doctor clinic?
RQ2	What constraints and mechanisms are required to prevent appointment time conflicts at both application and database levels?
RQ3	How can a normalised schema (3NF) minimise data redundancy while preserving referential integrity across entities?
RQ4	To what extent can aggregated SQL queries provide actionable clinical and financial analytics in real time?
RQ5	How can a three-tier web architecture (React + Node.js + MySQL) effectively serve as a scalable clinic management solution?

Main Entities Overview

PATIENT	DOCTOR	DEPARTMENT	CLINIC	APPOINTMENT
Stores demographic info: name, phone, address, birth_date, job	Linked to a department; holds contact and speciality data	20 medical specialities; parent entity for doctors & clinics	Physical locations; belongs to one department	Central associative entity; links Patient, Doctor & Clinic

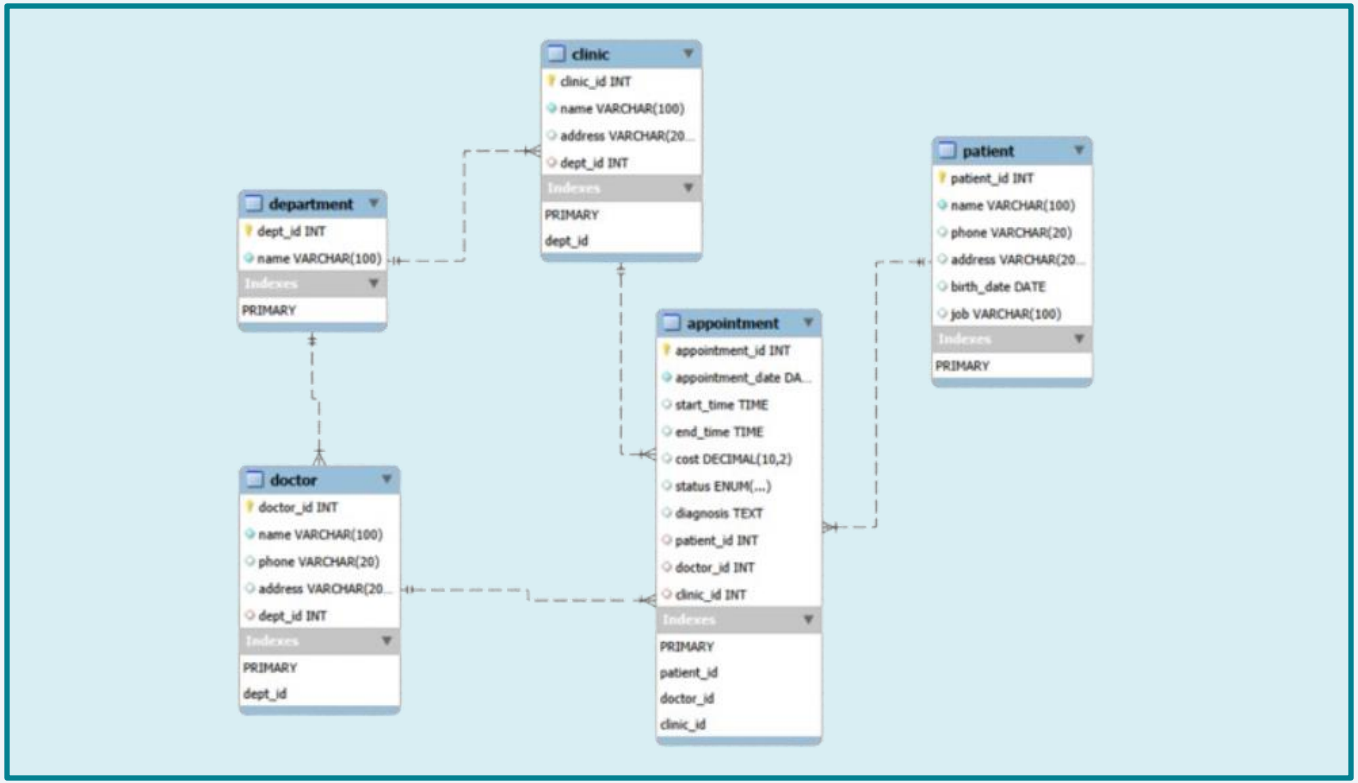
APPOINTMENT is the central associative entity — it connects PATIENT, DOCTOR, and CLINIC.

Entity Relationships

Relationship	Entity A	Entity B	Cardinality	Participation
BELONGS_TO	DOCTOR	DEPARTMENT	N : 1	Partial / Total
BELONGS_TO	CLINIC	DEPARTMENT	N : 1	Partial / Total
BOOKED_FOR	APPOINTMENT	PATIENT	N : 1	Total / Partial
CONDUCTED_BY	APPOINTMENT	DOCTOR	N : 1	Total / Partial
HELD_AT	APPOINTMENT	CLINIC	N : 1	Partial / Partial

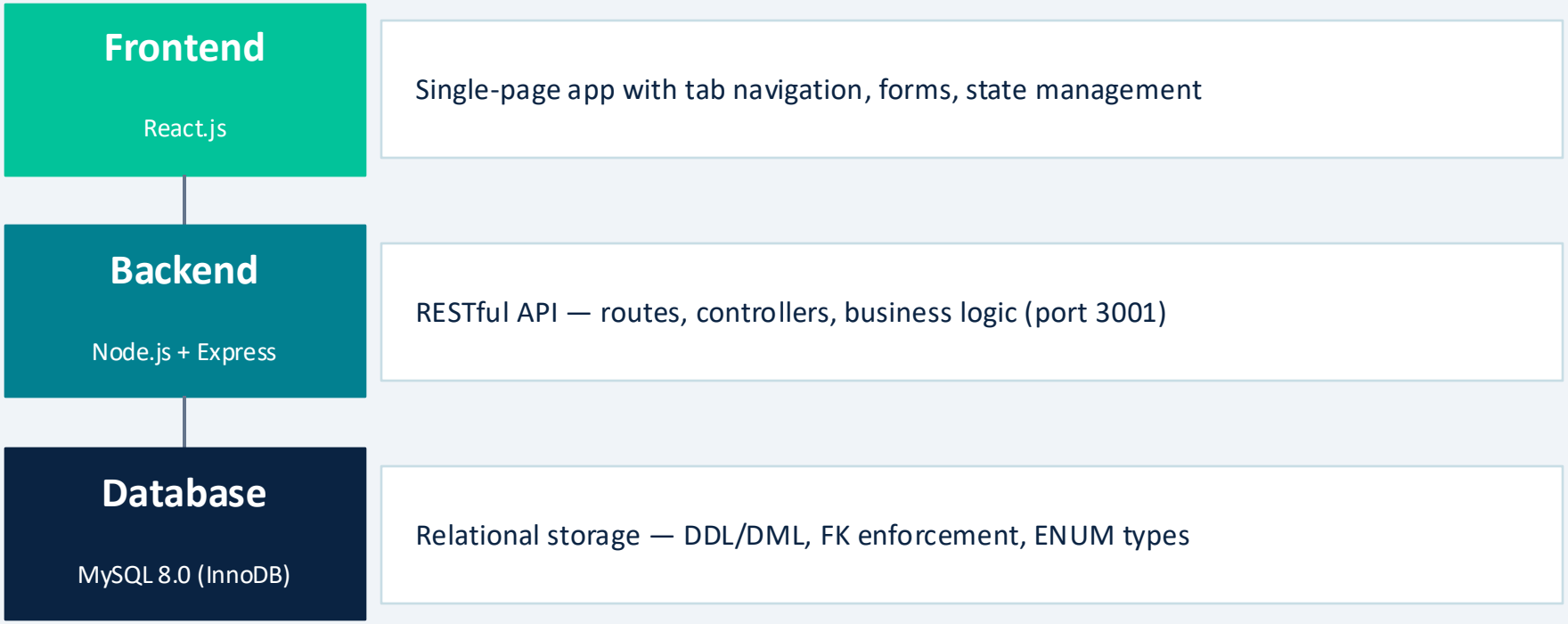
- All five entities have surrogate integer primary keys — no natural key dependencies
- APPOINTMENT is both a strong entity (has its own PK) and an associative junction entity

Entity-Relationship Diagram



Key: PK = Primary Key | FK = Foreign Key | → = References

System Architecture



Communication: Browser fetch API → Express Routes → mysql2 driver → MySQL InnoDB

Technologies Used

Database

- MySQL 8.0
- InnoDB engine
- utf8mb4 charset
- ENUM, DECIMAL types

Backend

- Node.js runtime
- Express.js framework
- mysql2 driver
- CORS middleware

Frontend

- React.js (SPA)
- useState / useEffect hooks
- fetch API
- Custom CSS (Flexbox/Grid)

Dev Tools

- MySQL Workbench
- Postman (API testing)
- npm package manager

Backend API Structure

RESTful API Endpoints

GET	*/patients, /doctors, /clinics, /departments, /appointments*
POST	*/patients, /doctors, /clinics, /departments, /appointments*
DELETE	*/patients/:id, /doctors/:id, /clinics/:id, /departments/:id*
GET	*/analytics/diagnosis-trend	/analytics/revenue*

Key Logic

- Parameterised queries (mysql2) prevent SQL injection
- Overlap check query fires BEFORE every appointment INSERT
- Global CORS middleware; single persistent DB connection (academic scope)

Database Tables Overview

Table	Primary Key	Foreign Keys	Key Constraint
DEPARTMENT	dept_id	—	name NOT NULL
PATIENT	patient_id	—	name NOT NULL
DOCTOR	doctor_id	dept_id → DEPARTMENT	name NOT NULL
CLINIC	clinic_id	dept_id → DEPARTMENT	name NOT NULL
APPOINTMENT	appointment_id	patient_id, doctor_id, clinic_id	date NOT NULL; status ENUM

InnoDB engine enforces all FK constraints; RESTRICT default on DELETE/UPDATE.

Constraints & Normalisation

- Entity Integrity: PKs are INT NOT NULL — MySQL auto-enforces uniqueness
- Referential Integrity: InnoDB FK constraints prevent orphaned records
- Domain Constraints: ENUM status, DATE NOT NULL, DECIMAL(10,2) for cost
- Business Constraint: Overlap check in API before INSERT
- Name NOT NULL on all core tables — no anonymous records

- 1NF: All columns hold atomic values; no repeating groups
- 2NF: Single-column PKs eliminate partial dependencies
- 3NF: No transitive dependencies — dept stored as FK, not text string
- Surrogate integer PKs: faster joins, storage efficiency, stability
- ENUM internally stored as 1-2 byte integer — efficient & self-documenting

SQL Implementation Highlights

DDL — Table Creation

- ENGINE=InnoDB for FK enforcement
- utf8mb4 charset — full Unicode support
- Secondary indexes on all FK columns (auto-created) for JOIN performance
- FOREIGN KEY … REFERENCES with RESTRICT default (no cascading deletes)

DML — Data Manipulation

- Bulk INSERT with FOREIGN_KEY_CHECKS=0 during seeding
- 20 departments, 20 patients, 20 doctors, 10+ clinics loaded
- DELETE enforced by RESTRICT — appointments protect patient/doctor records
- UPDATE logic handled via parameterised API calls

All queries use mysql2 parameterised placeholders (?) — SQL injection is fully prevented.

Proposed Views & Triggers

Proposed SQL Views

- vw_appointment_detail — full JOIN across all 5 tables for appointment summaries
- vw_doctor_appointment_count — workload & revenue per doctor
- vw_diagnosis_summary — frequency, avg cost, first/last seen per diagnosis

Proposed Triggers

- trg_no_overlap — BEFORE INSERT: database-level overlap check using SIGNAL SQLSTATE
- trg_auto_complete_status — BEFORE UPDATE: auto-completes past appointments
- trg_appointment_audit — AFTER DELETE: logs every deleted appointment with timestamp

Note: Views and triggers are proposed enhancements; the current dump does not include them.

Preventing Overlapping

Overlap (FAIL)

10:00 ——— 11:00 10:30 ——— 11:30 conflict

No overlap (SUCCESS)

10:00 ——— 11:00 11:30 —— 12:00 safe

Application layer

```
-- Node.js backend executes this before every INSERT:
SELECT * FROM Appointment
WHERE  doctor_id        = ?              -- same
doctor
AND    appointment_date = ?              -- same date
AND (  start_time       < ?              -- existing
starts before new ends
   AND end_time         > ?  );          -- existing
ends after new starts
-- If result.length > 0 → reject with HTTP 400
```

Database Layer

```
DELIMITER $$
CREATE TRIGGER trg_no_overlap
BEFORE INSERT ON appointment
FOR EACH ROW
BEGIN
    DECLARE v_count INT DEFAULT 0;
    SELECT COUNT(*) INTO v_count
    FROM   appointment
    WHERE  doctor_id        = NEW.doctor_id
    AND    appointment_date = NEW.appointment_date
    AND    start_time       < NEW.end_time
    AND    end_time         > NEW.start_time;
    IF v_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Time slot already booked for this
doctor.';
    END IF;
END$$
DELIMITER ;
```

Overlapping

Patient Information with Appointment

Question 2— Patient Query

patient_id	patient_name	phone	birth_date	job	appointment_id	appointment_date	start_time	end_time	status	diagnosis	cost	doctor_name	department
12501	Patient_1	01020194829	1999-04-19	Designer	18	2025-09-25	11:00:00	14:30:00	completed	migraine	355.76	Dr. Doctor_12	Rheumatology
12501	Patient_1	01020194829	1999-04-19	Designer	12	2024-05-31	12:00:00	14:30:00	in progress	diabetes	716.16	Dr. Doctor_2	Gastroenterology
12502	Patient_2	01067107704	1986-04-19	Designer	6	2024-12-08	12:00:00	13:30:00	in progress	diabetes	235.32	Dr. Doctor_11	Emergency
12503	Patient_3	01073356192	1973-04-19	Engineer	3	2024-03-07	11:00:00	16:30:00	scheduled	heart disease	530.19	Dr. Doctor_8	Emergency
12504	Patient_4	01026587589	1977-04-19	Doctor	2	2024-09-18	15:00:00	09:30:00	completed	fatty liver	528.21	Dr. Doctor_10	Ophthalmology
12505	Patient_5	01076390074	1992-04-19	Designer	14	2025-01-15	10:00:00	11:30:00	scheduled	checkup	706.28	Dr. Doctor_5	Nephrology
12505	Patient_5	01076390074	1992-04-19	Designer	5	2024-03-18	08:00:00	16:30:00	in progress	fatty liver	558.08	Dr. Doctor_13	Pulmonology
12506	Patient_6	01013341232	1984-04-19	Designer	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12507	Patient_7	01014766721	1996-04-19	Doctor	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12508	Patient_8	01031124553	1974-04-19	Doctor	17	2025-04-12	08:00:00	11:30:00	postponed	checkup	614.22	Dr. Doctor_18	Immunology
12508	Patient_8	01031124553	1974-04-19	Doctor	4	2024-08-20	14:00:00	12:30:00	postponed	migraine	668.13	Dr. Doctor_12	Rheumatology
12508	Patient_8	01031124553	1974-04-19	Doctor	10	2023-09-26	12:00:00	15:30:00	completed	migraine	586.55	Dr. Doctor_10	Ophthalmology
12509	Patient_9	01041263008	1969-04-19	Designer	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12510	Patient_10	01050677858	1984-04-19	Teacher	13	2025-07-25	09:00:00	15:30:00	scheduled	checkup	787.50	Dr. Doctor_20	Hematology
12511	Patient_11	01080127608	1980-04-19	Engineer	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12512	Patient_12	01021906688	1973-04-19	Student	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12513	Patient_13	01059089778	1984-04-19	Engineer	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL
12514	Patient_14	01015701863	2001-04-19	Teacher	9	2025-03-02	09:00:00	14:30:00	postponed	diabetes	958.25	Dr. Doctor_8	Emergency
12515	Patient_15	01040725232	1994-04-19	Teacher	11	2025-04-25	10:00:00	12:30:00	completed	diabetes	364.27	Dr. Doctor_20	Hematology
12515	Patient_15	01040725232	1994-04-19	Teacher	7	2025-03-27	10:00:00	13:30:00	postponed	diabetes	722.19	Dr. Doctor_13	Pulmonology
12515	Patient_15	01040725232	1994-04-19	Teacher	16	2025-03-13	14:00:00	16:30:00	completed	diabetes	486.25	Dr. Doctor_19	Gastroenterology
12515	Patient_15	01040725232	1994-04-19	Teacher	15	2025-01-04	13:00:00	09:30:00	completed	heart disease	364.54	Dr. Doctor_15	Dermatology

```
SELECT
    p.patient_id,
    p.name
AS patient_name,
    p.phone,
    p.birth_date,
    p.job,
    a.appointment_id,
    a.appointment_date,
    a.start_time,
    a.end_time,
    a.status,
    a.diagnosis,
    a.cost,
    d.name          AS doctor_name,
    dep.name        AS department
FROM      patient     p
LEFT JOIN appointment a   ON p.patient_id  = a.patient_id
LEFT JOIN doctor      d   ON a.doctor_id   = d.doctor_id
LEFT JOIN department  dep ON d.dept_id     = dep.dept_id
ORDER BY  p.patient_id, a.appointment_date DESC;
```

JOIN Logic Explained

LEFT JOIN throughout
Patients with NO appointments still appear (NULL columns). INNER JOIN would exclude them.

p → a (patient_id)
Links each patient to all their appointment records.

a → d (doctor_id)
Resolves doctor name from the doctor table.

d → dep (dept_id)
Resolves department name via doctor's dept_id.

ORDER BY
Groups by patient, then sorts appointments newest-first.

UPCOMING APPOINTMENT COUNT

Question 3— APPOINTMENT

```
CREATE OR REPLACE VIEW vw_upcoming_appointments AS
SELECT
    d.doctor_id,
    d.name                          AS doctor_name,
    dep.name                        AS department,
    COUNT(a.appointment_id)         AS upcoming_count
FROM      doctor      d
LEFT JOIN appointment a
    ON  a.doctor_id         = d.doctor_id
    AND a.appointment_date BETWEEN CURDATE()

 AND DATE_ADD(CURDATE(), INTERVAL 1 MONTH)
    AND a.status
IN ('scheduled', 'in progress')
LEFT JOIN department  dep ON
d.dept_id = dep.dept_id
GROUP BY  d.doctor_id, d.name, dep.name
ORDER BY  upcoming_count DESC;
```

doctor_id	doctor_name	department	upcoming_count
1001	Dr. Doctor_1	General Surgery	0
1002	Dr. Doctor_2	Gastroenterology	0
1003	Dr. Doctor_3	Cardiology	0
1004	Dr. Doctor_4	Gastroenterology	0
1005	Dr. Doctor_5	Nephrology	0
1006	Dr. Doctor_6	Hematology	0
1007	Dr. Doctor_7	Pulmonology	0
1008	Dr. Doctor_8	Emergency	0
1009	Dr. Doctor_9	Rheumatology	0
1010	Dr. Doctor_10	Ophthalmology	0
1011	Dr. Doctor_11	Emergency	0
1012	Dr. Doctor_12	Rheumatology	0
1013	Dr. Doctor_13	Pulmonology	0
1014	Dr. Doctor_14	Urology	0
1015	Dr. Doctor_15	Dermatology	0
1016	Dr. Doctor_16	Rheumatology	0
1017	Dr. Doctor_17	Cardiology	0
1018	Dr. Doctor_18	Immunology	0

GROUP BY & Aggregation Logic

GROUP BY

Collapses all appointment rows per doctor into one summary row.

COUNT(a.appointment_id)

Counts non-NULL values — doctors with 0 upcoming return 0, not excluded.

LEFT JOIN on 3 ON conditions

Date BETWEEN, status IN, doctor_id all in ON — unmatched doctors show count 0.

CURDATE() + INTERVAL 1 MONTH

Restricts to next 30 days dynamically — no hardcoded dates.

ORDER BY DESC

Sorts from busiest to least busy — ideal for workload management.

System Workflow — End-to-End

1. System Initialisation — Seed DEPARTMENT (20 specialities) and link CLINIC records
2. Doctor Registration — Staff add doctors via dashboard; linked to dept_id
3. Patient Registration — Receptionist captures demographics via Add Patient form
4. Appointment Booking — Overlap check fires; slot confirmed → status = 'scheduled'
5. Appointment Execution — Doctor updates status to 'in progress', records diagnosis
6. Completion & Analytics — Status set to 'completed'; revenue & trend data auto-updated
7. Record Maintenance — RESTRICT FK prevents deletion of active patient/doctor records

Appointment Booking — Deep Dive

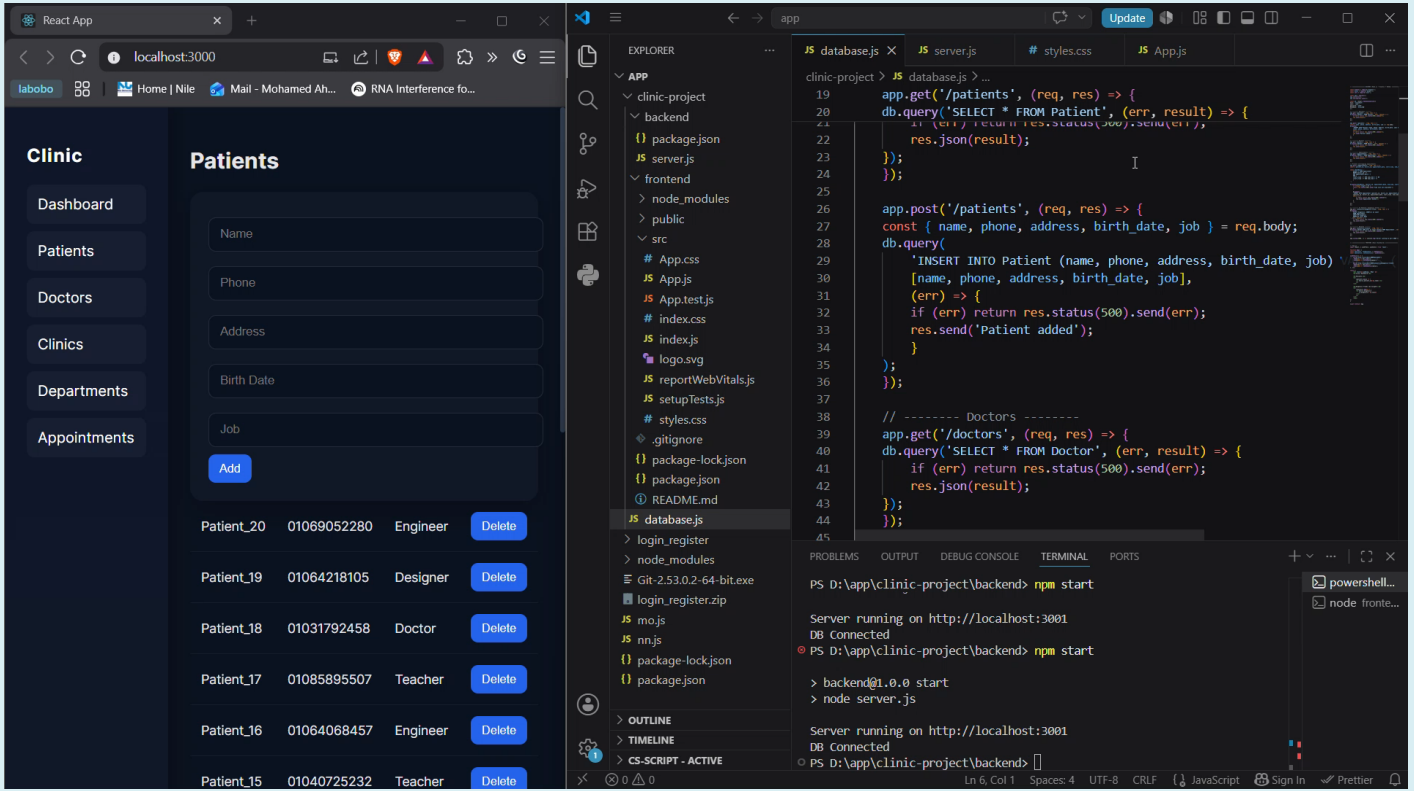
1	Receptionist selects patient & doctor IDs, enters date/time
2	React frontend sends POST /appointments with JSON body
3	Express API executes overlap-check query against appointment table
4a	Conflict found → HTTP 400 returned; booking rejected
4b	No conflict → INSERT executed; status = 'scheduled'
5	Frontend calls loadAll() — dashboard & table refresh instantly

Testing & Validation Overview

20	20	20	10+	20+
Departments seeded	Patient records	Doctor records	Clinic entries	Appointments

- All 4 appointment status values tested: scheduled, in progress, postponed, completed
- Multiple diagnoses repeated (e.g. diabetes in 6 records) — validates GROUP BY analytics
- Two clinics share dept_id=1 (Cardiology) — confirms 1:N department relationship
- Overlap rejection tested: doctor assigned two conflicting slots → HTTP 400 returned
- FK RESTRICT tested: deleting a patient with appointments raises InnoDB error

Live System Demonstration



CONCLUSION

Achievements & System Strengths

- 3NF-normalised schema — no redundancy, no update anomalies
- InnoDB FK constraints enforce referential integrity at DB level
- Parameterised queries throughout — SQL injection prevented
- Appointment overlap logic at both application and database levels
- Analytics endpoints deliver real business value (trends, revenue)
- Clean three-tier separation: database, API, frontend, presentation
- Dark-themed, responsive React dashboard with live data refresh
- ENUM type enforces appointment lifecycle without extra tables
- 20 departments and 20+ test records validate all relationships
- Scalable architecture: pool, pagination, JWT are natural next steps

Limitations & Future Improvements

CONCLUSION

- Current Limitations
- No user authentication (JWT not yet implemented)
- Single DB connection — not production-ready (no pooling)
- Frontend fetches full tables on every action (no pagination)
- No UPDATE endpoints for editing existing records
- Credentials hardcoded — should use .env variables

- Future Enhancements
- Add JWT-based authentication to secure all API endpoints
- Migrate to mysql.createPool for concurrent request handling
- Implement proposed views and triggers (Sections 12–13)
- Add server-side pagination and real-time search/filter
- Introduce UNIQUE constraints on phone/national ID fields

Thank You

Clinic Management System

We are happy to answer any questions from the jury.